

Analyses: Classification with MLP

Description

Classification of the feature vectors per character sequence (loaded from `~/results/features_combined.csv`) using Multi-Layer-Perceptrons (MLP), also known as feed-forward neural networks. The input layer consists of four nodes which take the estimated feature values per sequence, i.e. TTR, unigram entropy, entropy rate, and repetition rate. The hyperparameters are largely adopted from an earlier study (Bentz 2023). 100 random architectures in terms of hidden size (number of nodes), and depth (number of layers) are tested. See also visualization of one example architecture below.

```
knitr::include_graphics("~/Github/PaleoSigns/figures_external/mlpExample.png")
```

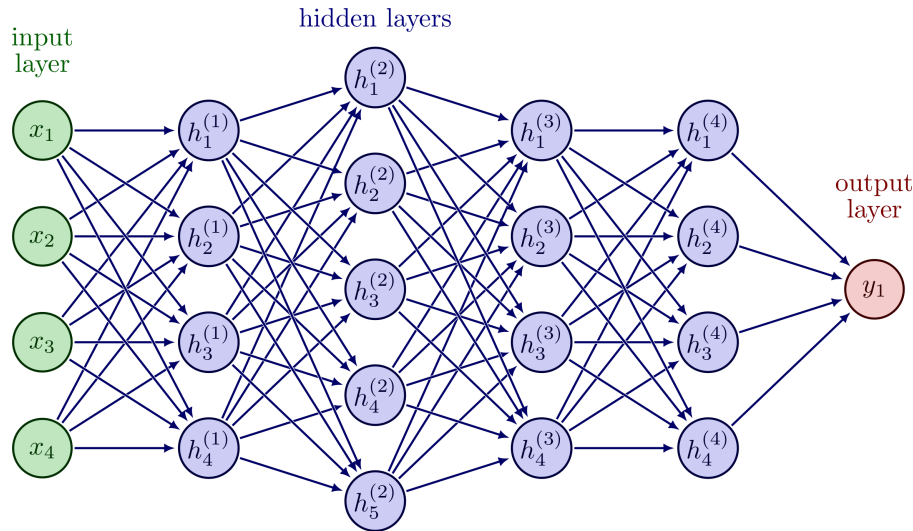


Figure 1: Example of one of the more complex architectures. This includes four hidden layers of four and five hidden units.

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"  
## [1] "x86_64-pc-linux-gnu"
```

Load Packages

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`. For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the `install_github()` function is used.

```
# install the latest version of neuralnet with bug fixes: devtools::install_github("bips-hb/neuralnet")
library(neuralnet)
library(sigmoid)
library(caret)
library(dplyr)
```

Give the package versions.

```
##      dplyr      caret    lattice    ggplot2    sigmoid neuralnet
##  "1.1.4"  "6.0-94"  "0.22-5"   "3.5.1"   "1.4.0"   "1.44.2"
```

Load Data

Load data table with estimated feature values per sequence.

```
# load feature value estimations
estimations.df <- read.csv("~/Github/PaleoSigns/results/features/features_combined.csv")
head(estimations.df)
```

```
##      id    subcorpus      sequence num.tokens num.types huni.chars
## 1 bst0001 Aurignacien vvvvvvvvvvvvvvvvvv      18        1 0.0000000
## 2 gkl0001 Aurignacien    ||| //|||_V      12        5 1.8962406
## 3 gkl0002 Aurignacien      vvvvv        5        1 0.0000000
## 4 gkl0003 Aurignacien      vvvv vvvv        9        2 0.5032583
## 5 gkl0004 Aurignacien vvvvvvvvvvvvvvvv      14        1 0.0000000
## 6 gkl0005 Aurignacien    |||            3        1 0.0000000
##  hrate.chars ttr.chars  rm.chars
## 1  0.6171397 0.05555556 1.0000000
## 2  1.5295563 0.41666667 0.6363636
## 3  0.5711903 0.20000000 1.0000000
## 4  0.9060047 0.22222222 0.7500000
## 5  0.6379550 0.07142857 1.0000000
## 6  0.4308271 0.33333333 1.0000000
```

Choose subcorpora for binary classification.

```
# choose subcorpora to compare: "Aurignacien", "TeDDi", "Cuneiform (Uruk V)",
# "Cuneiform (Uruk IV)", "Cuneiform (Uruk III)".
subcorpus1 <- "Aurignacien"
subcorpus2 <- "Cuneiform (Uruk V)"
selected <- c(subcorpus1, subcorpus2)
estimations.subset <- estimations.df[estimations.df$subcorpus %in% selected, ]

# downsample to have the same number of sequences across different combinations of corpora
# comment this out to use all the sequences (unbalanced across combinations of corpora)
# set.seed(02152018)
# estimations.subset <- sample_n(estimations.subset, 300)
```

```
# exclude sequences under a certain length
# estimations.subset <- estimations.subset[estimations.subset$num.tokens > 10, ]
```

Remove NAs (whole row)

```
estimations.subset <- na.omit(estimations.subset)
```

Center and scale the data

```
# first four columns are meta information, the others are the feature values
# be aware that this needs to be changed in case of other table format
estimations.scaled <- cbind(estimations.subset[, c("id", "subcorpus", "sequence",
                                                  "num.tokens", "num.types")],
                           scale(estimations.subset[, c("huni.chars", "hrate.chars",
                                                         "ttr.chars", "rm.chars")]))
```

Create Training and Test Sets

```
# Generating seed
set.seed(09012020)
# Randomly generating our training and test samples with a respective ratio of 2/3 and 1/3
datasample <- sample(2, nrow(estimations.scaled), replace = TRUE, prob = c(0.67, 0.33))
# Generate training set (select rows randomly by using datasample vector)
train <- estimations.scaled[datasample == 1, 1:ncol(estimations.scaled)]
# Generate test set
test <- estimations.scaled[datasample == 2, 1:ncol(estimations.scaled)]
# number of data points in test set
test.n <- nrow(test)
```

Implement MLP classifier

This is partly based on code given at http://uc-r.github.io/ann_classification (last accessed 18.01.2023). The activation function (tanh), error function (SSE), and backpropagation algorithm (rprop+) are here chosen according to the best model performance for sequences of 100 characters reported in Bentz (2023).

```
# Generate list of hidden unit architectures (number of layers and units)
# Choose number of random architectures
arch.no <- 100
# initialize empty list of hidden unit architectures
hidden.list <- vector(mode = "list", length = arch.no)
# set seed for reproducible results
set.seed(09012020)
# run loop to generate number of layers and hidden units randomly
for (i in 1:arch.no) {
  layer.no <- round(runif(1, min = 1, max = 4), 0)
  hidden.units <- round(runif(layer.no, min = 1, max = 5), 0)
  hidden.list[[i]] <- hidden.units
}
# remove duplicate hidden structures, i.e. only use "unique" vectors of numbers
```

```

hidden.list <- unique(hidden.list)

# Manual list of architectures for testing
# hidden.list <- list(c(4,4))

# initialize dataframe to append results to
results.df <- data.frame(hidden.structure = character(0),
                          hidden.size = numeric(0), hidden.depth = numeric(0),
                          err.fct = character(0), act.fct.name = character(0),
                          algorithm = character(0), baseline = numeric(0),
                          accuracy = numeric(0), accuracy.pvalue = numeric(0),
                          precision = numeric(0), recall = numeric(0),
                          f1 = numeric(0), test.n = numeric(0)
                          )

# set random seed for reproducibility
set.seed(123)
# counter
counter <- 0
# start time
start_time <- Sys.time()
for (hidden in hidden.list) {
  # Training
  # choose error function
  err.fct = 'sse' # options: 'sse', 'ce'
  # choose activation function
  act.fct = 'tanh' # options: 'logistic', 'tanh', 'softplus', 'relu',
  # beware: the last two options have to be written without quotes ''
  # note: softplus, relu, and tanh only work properly with sse as err.fct
  act.fct.name = 'tanh' # make sure to give name as character string (for later storage)
  # choose algorithm
  algorithm = 'rprop+' # options: 'rprop+', 'rprop-', 'backprop', 'sag', 'slr'
  # train classifier with neuralnet() function
  try({ # if the processing fails for a certain file, there will be no output for this file,
  # but the try() function allows the loop to keep running
  classifier.mlp <- neuralnet(subcorpus == subcorpus1 ~ .,
                             data = train[, c("subcorpus",
                                                "huni.chars",
                                                "hrate.chars",
                                                "ttr.chars",
                                                "rm.chars")],
                             hidden = hidden, # hidden layer architecture
                             threshold = 0.1, # defaults to 0.01
                             rep = 1, # number of reps in which new initial values are used,
                             # (essentially the same as a for loop)
                             stepmax = 100000, # defaults to 100K
                             linear.output = FALSE,
                             algorithm = algorithm, # defaults to "rprop+",
                             # i.e. resilient backpropagation
                             # learningrate = 0.1,
                             err.fct = err.fct,
                             act.fct = act.fct,
                             likelihood = TRUE,

```

```

        lifesign = 'minimal')
# classifier.mlp
# results matrix (each column represents one repetition)
# classifier.mlp$result.matrix

# Prediction
# Get prediction using the predict() function
mlp.predictions <- predict(classifier.mlp, test,
                           rep = which.min(classifier.mlp$result.matrix[1,]),
                           # Predict response values
                           # based on the "best" repetition (epoche),
                           # i.e. the one with the lowest error.
                           all.units = FALSE)
# assign a label according to the rule that the label is the label of subcorpus1
# if the prediction probability is >0.5, else assign label of subcorpus2.
mlp.predictions.rd <- ifelse(mlp.predictions > 0.5, subcorpus1, subcorpus2)

# create data frame with actual labels in the test set and predicted labels
mlp.predictions.df <- cbind(test, mlp.predictions.rd)

# save predictions for post hoc analyses
write.csv(mlp.predictions.df, file =
          paste(c("~/Github/PaleoSigns/results/classifier/MLP/predictions/predictions_MLP_",
                  subcorpus1, "_",
                  subcorpus2, "_",
                  paste("hidden",
                        paste(unlist(hidden),
                              collapse = ""),
                        sep = ""), ".csv"),
          collapse = ""),
          row.names = F)

# Model Evaluation
# creating a dataframe from known (true) test labels
test.labels <- data.frame(test$subcorpus)
# combining predicted and known classes
class.comparison <- data.frame(mlp.predictions.rd, test.labels)
# giving appropriate column names
names(class.comparison) <- c("predicted", "observed")
# inspecting our results table
head(class.comparison)
# get confusion matrix
cm <- confusionMatrix(as.factor(class.comparison$predicted),
                      reference = as.factor(class.comparison$observed))
#print(cm)
# get performance metrics from the output list of confusionMatrix()
baseline <- cm$overall['AccuracyNull']
accuracy <- cm$overall['Accuracy']
accuracy.pvalue <- cm$overall['AccuracyPValue']
f1 <- cm[["byClass"]][["F1"]]
recall <- cm[["byClass"]][["Recall"]]
precision <- cm[["byClass"]][["Precision"]]

```

```

# unname and round resulting values (except for pvalue,
# since this needs to be corrected later)
baseline <- round(unname(baseline), 2)
accuracy <- round(unname(accuracy), 2)
f1 <- round(unname(f1), 2)
recall <- round(unname(recall), 2)
precision <- round(unname(precision), 2)

# append results to dataframe
hidden.structure <- paste(unlist(hidden), collapse = ",")
hidden.size <- sum(unlist(hidden))
hidden.depth <- length(hidden)
local.df <- data.frame(hidden.structure, hidden.size,
                       hidden.depth, err.fct, act.fct.name, algorithm,
                       baseline, accuracy, accuracy.pvalue,
                       precision, recall, f1, test.n)
results.df <- rbind(results.df, local.df)
})
counter <- counter + 1
print(counter)
}

```

```

## [1] 1
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
## [1] 11
## [1] 12
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 13
## [1] 14
## [1] 15
## [1] 16
## [1] 17
## [1] 18
## [1] 19
## [1] 20
## [1] 21
## [1] 22
## [1] 23
## [1] 24
## [1] 25
## [1] 26
## [1] 27
## [1] 28

```

```

## [1] 29
## [1] 30
## [1] 31
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 32
## [1] 33
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 34
## [1] 35
## [1] 36
## [1] 37
## [1] 38
## [1] 39
## [1] 40
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 41
## [1] 42
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 43
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 44
## [1] 45
## [1] 46
## [1] 47
## [1] 48
## [1] 49
## [1] 50
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 51
## [1] 52
## [1] 53
## [1] 54
## [1] 55
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 56
## [1] 57
## [1] 58
## [1] 59
## [1] 60
## [1] 61
## [1] 62
## Error in cbind(1, pred) %*% weights[[num_hidden_layers + 1]] :
##   requires numeric/complex matrix/vector arguments
## [1] 63
## [1] 64
## [1] 65
## [1] 66

```

```
## [1] 67
## [1] 68
## [1] 69
## [1] 70
## [1] 71
## [1] 72
## [1] 73
## [1] 74

end_time <- Sys.time()
proc.time <- end_time - start_time
#print(proc.time)
```

Write performance metrics to file.

```
# classification results summaries
write.csv(results.df,
          file = paste(c("~/Github/PaleoSigns/results/classifier/MLP/performance/performance_MLP_",
                        subcorpus1, "_", subcorpus2, ".csv"), collapse = ""),
          row.names = F)
```

Visualize the NN

Visualize the NN with the best weights after training. Note this is the *last run* of the for loop through different architectures.

```
#mlp.plot <- plot(classifier.mlp, rep = 'best', show.weights = F)
#mlp.plot
```

Package References

```
## Fritsch S, Guenther F, Wright M (2019). _neuralnet: Training of Neural
## Networks_. R package version 1.44.2,
## <https://CRAN.R-project.org/package=neuralnet>.

## Quast B (2022). _sigmoid: Sigmoid Functions for Machine Learning_. R
## package version 1.4.0, <https://CRAN.R-project.org/package=sigmoid>.

## Kuhn, Max (2008). "Building Predictive Models in R Using the caret
## Package." _Journal of Statistical Software_, *28*(5), 1-26.
## doi:10.18637/jss.v028.i05 <https://doi.org/10.18637/jss.v028.i05>,
## <https://www.jstatsoft.org/index.php/jss/article/view/v028i05>.
```

Literature References

Bentz, Christian (2023). The Zipfian Challenge: Learning the statistical fingerprint of natural languages. CoNLL, Singapore. <https://aclanthology.org/2023.conll-1.3/>